JDBC & ADO .NET

Seminar 1

Cuprins

- Architectura aplicațiilor conectate la o bază de date
- JDBC API (Java)
- ADO.NET API (C#)
- Gestiunea conexiunilor
- Execuția interogărilor SQL
- Procesarea rezultatelor
- Interogări parametrizabile
- Gestiunea tranzacțiilor
- Bune practici legate de gestiunea resurselor

JDBC ADO.NET - Introducere

JDBC (Java Database Connectivity)

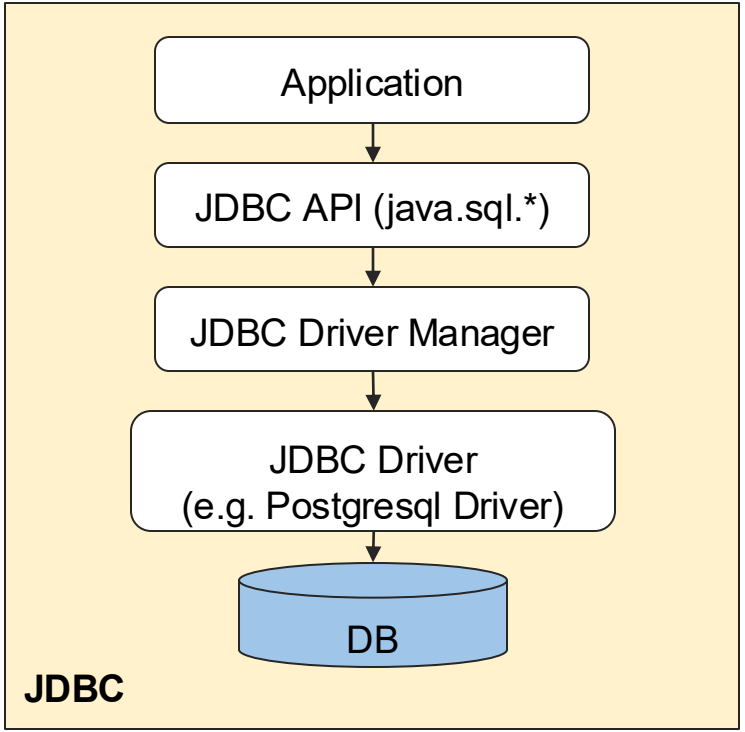
- API-ul standard în Java pentru conectarea la bazele de date
- Arhitectură bazată pe drivere
- Independent de platformă (DB)
- Funcționează cu orice BD compatibilă (care oferă un driver)

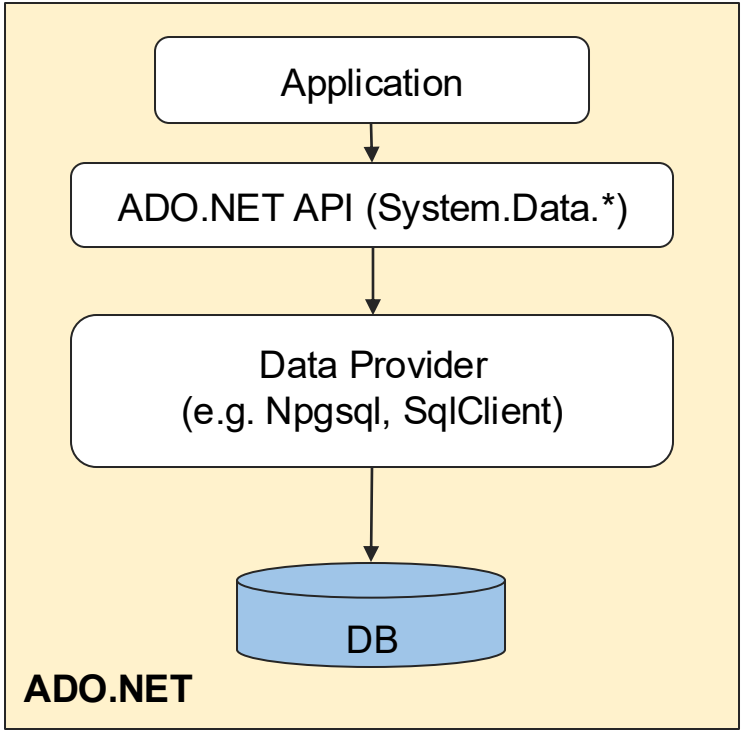
ADO.NET (ActiveX Data Objects .NET)

- Tehnologia de acces la date de la Microsoft în .NET
- Arhitectură bazată pe provider
- Suportă mai multe surse de date
- Integrată în framework-ul .NET

Scop comun: Abstractizarea operațiilor cu bazele de date printr-un API orientat obiect

Architectura JDBC și ADO.NET





Clasele de bază JDBC

- **Interfețele principale (java.sql package):**
  - **DriverManager** - Gestionează driverele JDBC
  - **Connection** - Reprezintă o conexiune la baza de date
  - **Statement** - Execută comenzi SQL
  - **PreparedStatement** - Comenzi SQL pre-compilate cu parametrii
  - **ResultSet** - Conține rezultatele interogărilor
  - **SQLException** - Gestiunea erorilor operațiunilor cu BD
- Toate sunt interfețe implementate de către driverele specifice fiecărui DB

Clase de bază ADO.NET

Clasele principale (namespace System.data):

- **DbConnection** - Clasa de bază pentru conexiunea la BD
  - NpgsqlConnection (PostgreSQL), SqlConnection (SQL Server), etc
- **DbCommand** - Execută comenzi SQL
  - NpgsqlCommand (PostgreSQL), SqlCommand (SQL Server), etc.
- **DbDataReader** - Folosită pentru citirea rezultatelor
  - NpgsqlDataReader (PostgreSQL), SqlDataReader (SQL Server), etc.
- **DbParameter** - parametrii pentru instrucțiuni SQL
- **DbException** - Gestiunea erorilor operațiunilor cu BD

Toate sunt clase abstracte cu implementări specifice fiecărui provider

Adăugarea driver-ului JDBC

Java - MAVEN (pom.xml):

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.0</version>
</dependency>
```

Java - Gradle (build.gradle):

```
implementation 'org.postgresql:postgresql:42.7.0'
```

C# - .NET CLI

```
dotnet add package Npgsql
```

Parametrii de conectare (Connection String)

Formatul JDBC:

jdbc:<dbvendor>://host:port/database?param1=value1&param2=value2

Exemplu JDBC:

```
String url = "jdbc:postgresql://localhost:5432/mydb?user=postgres&password=mypassword
```

Formatul ADO.NET:
Host=host;Port=port;Database=database;Username=user;Password=pass

Exemplu ADO.NET:

```
string connectionString =
"Host=localhost;Port=5432;Database=mydb;Username=postgres;Password=mypassword";
```

Stabilirea unei conexiuni JDBC

Conexiune simplă:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
	public static Connection getConnection() throws SQLException {
		String url = "jdbc:postgresql://localhost:5432/mydb";
		String username = "postgres";
		String password = "mypassword";
		return DriverManager.getConnection(url, username, password);
	}
}
```

Notă: Versiunile mai noi de JDBC nu mai necesită încarcarea explicita a driver-ului (Class.forName)

Stabilirea unei conexiuni ADO.NET

Conexiune simplă:

```
using Npgsql;
using System.Data;
public class DatabaseConnection {
	public static NpgsqlConnection GetConnection() {
		string connectionString =
	"Host=localhost;Port=5432;Database=mydb;Username=postgres;Password=mypassword";
		return new NpgsqlConnection(connectionString);
	}
}
```

Important: Obiectul Connection este creat, dar conexiunea nu este încă deschisă

Deschiderea și închiderea conexiunilor

JDBC - Try-with-resources (Recomandat):

```
try (Connection conn = DriverManager.getConnection(url, user, pass)) {
        // Use connection
        // Automatically closed at end of try block
} catch (SQLException e) {
        e.printStackTrace();
}
```

ADO.NET - Instrucțiunea using (Recomandat):

```
using (var conn = new NpgsqlConnection(connectionString)) {
        conn.Open();
        // Use connection
        // Automatically closed and disposed
}
```

Important: Ne asigurăm întotdeauna că închidem conexiunile folosind elementele de limbaj care garantează eliberarea resurselor

De ce este importantă gestionarea conexiunilor

Exemplu (Java):

```
// BAD CODE - Connection never closed
Connection conn = DriverManager.getConnection(url, user, pass);
ResultSet rs = conn.createStatement().executeQuery("SELECT * FROM users");
// If exception occurs, connection is leaked!
```

Consecințe:

- Baza de date consumă toate conexiunile disponibile
- Aplicația nu mai răspunde
- Necesita reponirea aplicației și/sau a bazei de date

Soluția:

- Intotdeauna folosim **try-with-resources** (Java) sau instrucțiunea **using** (C#)
- Ne asigurăm de închiderea resurelor în blocurile **finally** dacă nu utilizăm metoda de mai sus

Executarea unor instrucțiuni SQL simple - JDBC

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT id, name, email FROM employees")) {

        while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String email = rs.getString("email");
                System.out.println(id + ": " + name + " - " + email);
        }
} catch (SQLException e) {
        e.printStackTrace();
}
```

Notă: Exemplul este pentru o instrucțiunie SQL statică, fără parametrii

Metode ResultSet/DataReader

JDBC

```
rs.next() // Move to next row, returns false if no more rows
rs.getInt("columnName") // Get integer by column name
rs.getInt(1) // Get integer by column index (1-based)
rs.getString("columnName") // Get string
rs.getDouble("columnName") // Get double
rs.getDate("columnName") // Get date
rs.getBoolean("columnName") // Get boolean
rs.wasNull() // Check if last value was NULL
```

ADO.NET

```
reader.Read() // Move to next row, returns false if no more rows
reader.GetInt32(0) // Get integer by index (0-based)
reader.GetString(1) // Get string
reader.GetDouble(2) // Get double
reader.GetDateTime(3) // Get datetime
reader.GetBoolean(4) // Get boolean
reader.IsDBNull(0) // Check if value is NULL
reader["columnName"] // Get value by name (returns object)
```

SQL Injection - BIG PROBLEM

Java:

```
// DANGEROUS - SQL Injection vulnerability
String email = userInput;
String sql = "SELECT * FROM users WHERE email = '" + email + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

Exemplu de atac:

User Input: **' OR '1'='1**
Final SQL: **SELECT * FROM users WHERE email = '' OR '1'='1'**
Result: Returns ALL users!

Sau chiar mai rău:

User Input: **'; DROP TABLE users; --**
Final SQL: **SELECT * FROM users WHERE email = ''; DROP TABLE users; --'**
Result: Table deleted!

Soluția - Interogări (queries) parametrizate

De ce folosim interogări parametrizate ?

- Instrucțiunea SQL și date sunt gestionate separat
- Baza de date tratează parametrii ca și date, nu ca cod executabil (SQL)
- Previne atacurile de tip SQL injection
- De multe ori oferă beneficii de performanță (caching la nivel de instrucțiune)

Folosim PreparedStatement (JDBC) sau DbCommand cu parametrii (ADO.NET)

Interogări (queries) parametrizate - JDBC

Folosim **PreparedStatement**:

```
String sql = "SELECT * FROM employees WHERE department_id = ? AND salary > ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        // Set parameters (1-based indexing)
        pstmt.setInt(1, departmentId);
        pstmt.setDouble(2, minSalary);
        try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                        System.out.println(rs.getString("name"));
                }
        }
}
```

- Folosim ? pentru a marca un parametru
- Parametrii are indexati de la 1
- Metode setter cu verificarea tipul de data: setInt, setString, setDouble, etc.

Interogări (queries) parametrizate - ADO.NET

```
string sql = "SELECT * FROM employees WHERE department_id = @deptId AND salary > @minSalary";
using (var conn = new NpgsqlConnection(connectionString)) {
        conn.Open();
        using (var cmd = new NpgsqlCommand(sql, conn)) {
                // Add parameters (named parameters)
                cmd.Parameters.AddWithValue("@deptId", departmentId);
                cmd.Parameters.AddWithValue("@minSalary", minSalary);
                using (var reader = cmd.ExecuteReader()) {
                        while (reader.Read()) {
                                Console.WriteLine(reader.GetString(1));
                        }
                }
        }
}
```

- Folosim `@paramName` pentru a marca un parametru
- Parametrii pot fi adaugați în orice ordine
- Metoda **AddWithValue** nu există in clasa abstractă Command, doar in clasele NpgsqlCommand/SqlCommand

Operațiuni de INSERT - JDBC

Inserare date:

```
String sql = "INSERT INTO employees (name, email, salary, department_id) VALUES (?, ?, ?, ?)";
try (Connection conn = DriverManager.getConnection(url, user, pass);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, "John Doe");
        pstmt.setString(2, "john@example.com");
        pstmt.setDouble(3, 55000.00);
        pstmt.setInt(4, departmentId);
        int rowsAffected = pstmt.executeUpdate();
        System.out.println("Rows inserted: " + rowsAffected);
}
```

Obținerea cheilor primare generate automat:

```
PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
pstmt.executeUpdate();
try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
        if (generatedKeys.next()) {
                long id = generatedKeys.getLong(1);
                System.out.println("Generated ID: " + id);
} }
```

Operațiuni de INSERT - ADO.NET

Inserare date:

```
string sql = "INSERT INTO employees (name, email, salary, department_id) VALUES (@name, @email, @salary, @deptId)";
using (var conn = new NpgsqlConnection(connectionString)) {
        conn.Open();
        using (var cmd = new NpgsqlCommand(sql, conn)) {
                cmd.Parameters.AddWithValue("@name", "John Doe");
                cmd.Parameters.AddWithValue("@email", "john@example.com");
                cmd.Parameters.AddWithValue("@salary", 55000.00);
                cmd.Parameters.AddWithValue("@deptId", departmentId);
                int rowsAffected = cmd.ExecuteNonQuery();
                Console.WriteLine($"Rows inserted: {rowsAffected}");
} }
```

Obținerea cheilor primare generate automat:

```
string sql = "INSERT INTO employees (name, email, salary, department_id) VALUES (@name, @email, @salary, @deptId)
RETURNING id";
object result = cmd.ExecuteScalar();
long id = Convert.ToInt64(result);
```

Operațiuni de UPDATE

JDBC:

```
String sql = "UPDATE employees SET salary = ?, email = ? WHERE id = ?";
try (Connection conn = DriverManager.getConnection(url, user, pass); PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setDouble(1, newSalary);
        pstmt.setString(2, newEmail);
        pstmt.setInt(3, employeeId);
        int rowsAffected = pstmt.executeUpdate();
        System.out.println("Rows updated: " + rowsAffected);
}
```

ADO.NET:

```
string sql = "UPDATE employees SET salary = @salary, email = @email WHERE id = @id";
using (var conn = new NpgsqlConnection(connectionString)) {
        conn.Open();
        using (var cmd = new NpgsqlCommand(sql, conn)) {
                cmd.Parameters.AddWithValue("@salary", newSalary);
                cmd.Parameters.AddWithValue("@email", newEmail);
                cmd.Parameters.AddWithValue("@id", employeeId);
                int rowsAffected = cmd.ExecuteNonQuery();
                Console.WriteLine($"Rows updated: {rowsAffected}");
        }
}
```

Gestionarea valorilor NULL

```
// JAVA - Setting NULL values
if (email == null) {
        pstmt.setNull(1, java.sql.Types.VARCHAR);
} else {
        pstmt.setString(1, email);
}
// Java - Checking for NULL values
String email = rs.getString("email");
if (rs.wasNull()) {
        email = "N/A";
}
```

```
// C# - Setting NULL values
cmd.Parameters.AddWithValue("@email", email ?? (object)DBNull.Value);

// C# - Checking for NULL values
string email = reader.IsDBNull(2) ? "N/A" : reader.GetString(2);
```

Tranzacții - scurtă prezentare

Ce este o **Tranzacție**:

- O unitate logică de lucru
- Proprietățile ACID: Atomicitate, Consistență, Izolare, Durabilitate
- Atomicitate: sau sunt executate toate operațiunile sau niciuna

Exemplu:

Transfer de bani intre două conturi:

1. Scadem (debităm) 100 RON din contul A
2. Adăugam (creditătm) 100 RON în contul B

Ambele operațiuni trebuie să fie executate cu succes sau ambele vor fi anulate !

Comportamenul implicit:

- JDBC/ADO.NET folosesc modul **auto-commit**
- Fiecare instrucțiune SQL este o tranzacție separată

Tranzacții - JDBC - control manual

```
Connection conn = null;
try {
        conn = DriverManager.getConnection(url, user, pass);
        conn.setAutoCommit(false); // Start transaction // Execute multiple operations
        PreparedStatement pstmt1 = conn.prepareStatement( "UPDATE accounts SET balance = balance - ? WHERE id = ?");
        pstmt1.setDouble(1, 100.0);
                pstmt1.setInt(2, accountAId);
        pstmt1.executeUpdate();
        PreparedStatement pstmt2 = conn.prepareStatement( "UPDATE accounts SET balance = balance + ? WHERE id = ?");
        pstmt2.setDouble(1, 100.0);
        pstmt2.setInt(2, accountBId);
        pstmt2.executeUpdate();
        conn.commit(); // Success - commit all changes
} catch (SQLException e) {
        if (conn != null) {
                try { conn.rollback(); } // Error - undo all changes
                catch (SQLException ex) { ex.printStackTrace();}
        } e.printStackTrace();
} finally {
        if (conn != null) {
                try { conn.setAutoCommit(true); // Restore auto-commit
                conn.close();
        } catch (SQLException e) { e.printStackTrace(); } }
}
```

Tranzacții - ADO.NET - control manual

```
using (var conn = new NpgsqlConnection(connectionString)) {
        conn.Open();
        using (var transaction = conn.BeginTransaction()) {
        try {// Execute multiple operations
                using (var cmd1 = new NpgsqlCommand( "UPDATE accounts SET balance = balance - @amount WHERE id =
        @id", conn, transaction)) {
                        cmd1.Parameters.AddWithValue("@amount", 100.0);
                        cmd1.Parameters.AddWithValue("@id", accountAId);
                        cmd1.ExecuteNonQuery();
                }
                using (var cmd2 = new NpgsqlCommand( "UPDATE accounts SET balance = balance + @amount WHERE id =
                @id", conn, transaction)) {
                        cmd2.Parameters.AddWithValue("@amount", 100.0);
                        cmd2.Parameters.AddWithValue("@id", accountBId);
                        cmd2.ExecuteNonQuery();
                }
                transaction.Commit(); // Success - commit all changes
        } catch (Exception) {
                transaction.Rollback(); // Error - undo all changes throw;
        }
}}
```

Tratarea erorilor - JDBC

```
try {
        // Database operations
} catch (SQLException e) {
        System.err.println("SQL State: " + e.getSQLState());          // standardized by ISO/ANSI and Open Group (X/Open)
        System.err.println("Error Code: " + e.getErrorCode());        // specific to DB Vendor
        System.err.println("Message: " + e.getMessage());
        Throwable t = ex.getCause();
         while(t != null) { System.out.println("Cause: " + t); t = t.getCause();  }

        // Check for specific errors
        if (e.getSQLState().equals("23505")) {
                System.err.println("Duplicate key violation!");
        } else if (e.getSQLState().equals("23503")) {
                System.err.println("Foreign key violation!");
        } e.printStackTrace();
}
```

Valori SQL State comune:

- 23505: Unique constraint violation
- 23503: Foreign key violation
- 23502: NOT NULL violation
- 42P01: Table does not exist

Tratarea erorilor - ADO.NET

```
try {
        // Database operations
} catch (NpgsqlException ex) {
        Console.WriteLine($"Error Code: {ex.ErrorCode}");
        Console.WriteLine($"Message: {ex.Message}");
        // Check for specific PostgreSQL errors
        if (ex.SqlState == "23505") {
                Console.WriteLine("Duplicate key violation!");
        } else if (ex.SqlState == "23503") {
                Console.WriteLine("Foreign key violation!");
        }
} catch (Exception ex) {
        Console.WriteLine($"General error: {ex.Message}");
}
```

Sumar bune practici

- Întotdeauna închideți resursele
  - Folosiți **try-with-resources** (Java) sau **using**(C#)
  - Închideți obiectele **ResultSet**, **Statement** și **Connection**
- Folosiți instrucțiuni parametrizate
  - Niciodată nu concatenați input-ul de la utilizator în SQL
  - Folosiți PreparedStatement sau comenzi parametrizate
- Tratați excepțiile
  - Prindeți erorile de tip SQLException
  - Afișați mesaje de eroare concludente utilizatorilor
  - Logați erori detaliate pentru a facilitate depanarea
- Folosiți tranzacții când e necesar
  - Multiple operații corelate trebuie tratate atomic
  - Tratați cazurile de rollback la procesarea erorilor

Șablon (Pattern) recomandat pentru acces la BD

```
public class EmployeeDAO {                          // DAO = DataAccessObject
        private String connectionUrl;
        public EmployeeDAO(String url) { this.connectionUrl = url; }
        public List<Employee> getEmployeesByDepartment(int deptId) throws SQLException {
                List<Employee> employees = new ArrayList<>();
                String sql = "SELECT * FROM employees WHERE department_id = ?";
                try (Connection conn = DriverManager.getConnection(connectionUrl);
                        PreparedStatement pstmt = conn.prepareStatement(sql)) {

                        pstmt.setInt(1, deptId);
                        try (ResultSet rs = pstmt.executeQuery()) {
                                while (rs.next()) {
                                        Employee emp = new Employee();
                                        emp.setId(rs.getInt("id"));
                                        emp.setName(rs.getString("name"));
                                        emp.setEmail(rs.getString("email"));
                                        employees.add(emp);
                                }
                        }
                }
        return employees;
        }
        public void addEmployee(Employee emp) throws SQLException { // INSERT implementation }
        public void updateEmployee(Employee emp) throws SQLException { // UPDATE implementation }
        public void deleteEmployee(int id) throws SQLException { // DELETE implementation } }
```

Șablon (Pattern) recomandat pentru acces la BD

```
public class EmployeeDAO {
        private readonly string _connectionString;
        public EmployeeDAO(string connectionString) { _connectionString = connectionString; }
        public List<Employee> GetEmployeesByDepartment(int deptId) {
                var employees = new List<Employee>();
                string sql = "SELECT * FROM employees WHERE department_id = @deptId";
                using (var conn = new NpgsqlConnection(_connectionString)) {
                        conn.Open();
                        using (var cmd = new NpgsqlCommand(sql, conn)) {
                        cmd.Parameters.AddWithValue("@deptId", deptId);
                                using (var reader = cmd.ExecuteReader()) {
                                        while (reader.Read()) {
                                                var emp = new Employee {
                                                        Id = reader.GetInt32(0),
                                                        Name = reader.GetString(1),
                                                        Email = reader.GetString(2)
                                                };
                                                employees.Add(emp);
          }}}}
        return employees; }
        public void AddEmployee(Employee emp) { /* INSERT */ }
        public void UpdateEmployee(Employee emp) { /* UPDATE */ }
        public void DeleteEmployee(int id) { /* DELETE */ } }
```

Pași de implementare Lab1 - sugestie

1. Creați clasele Model
    a. Clasa parinte (e.g. Department)
    b. Clasa copil (e.g. Employee)
2. Creați layer-ul DAO
    a. Metoda pentru a obține toate înregistrările din tabela părinte
    b. Metoda pentru a obține înregistrările din tabela copil pentru un parent ID
    c. Metodele CRUD pentru entitățile copil
3. Implementați GUI
    a. Tabelul/Lista părinte
    b. Tabel/Lista copil
    c. Butoanele Add/Edit/Delete
    d. Formularul (from) pentru introducerea/editarea datelor
4. Conectați GUI-ul la DAO
    a. Încărcați datele atunci când se schimbă selecția
    b. Apelați metodele DAO la acționarea butoanelor
    c. Tratați erorile prin afișarea mesajelor corespunzătoare

Debugging Tips

Activarea logare SQL:

```
String url = "jdbc:postgresql://localhost:5432/mydb?loggerLevel=TRACE";                    // JDBC
// In appsettings.json or code
NpgsqlLogManager.Provider = new ConsoleLoggingProvider(NpgsqlLogLevel.Debug);   // C#
```

Erori comune:

- Parametrii de conectari incorect (Connection string)
- Firewall-ul blocheaza portul DB, sau portul nu este deschis (SQL Server)
- Permisiuni utilizator DB (PostgreSQL: pg_hba.conf)
- Tabele coloane denumite greșit
- etc..

Configurare - Java

Java - Properties File (config.properties):

```
db.url=jdbc:postgresql://localhost:5432/mydb
db.username=postgres
db.password=mypassword
```

Java - Încărcare configurare:

```
Properties props = new Properties();
props.load(new FileInputStream("config.properties"));
String url = props.getProperty("db.url");
String user = props.getProperty("db.username");
String pass = props.getProperty("db.password");
```

Configurare - C#

C# - appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Database=mydb;Username=postgres;Password=mypass"
  }
}
```

C3 - Încărcare configurare:

```
var configuration = new ConfigurationBuilder() .AddJsonFile("appsettings.json") .Build();
```

Resurse și referințe

- **Documentație JDBC:**
  - Official Java Tutorial: https://docs.oracle.com/javase/tutorial/jdbc/
  - PostgreSQL JDBC Driver: https://jdbc.postgresql.org/documentation/
  - API Documentation: https://docs.oracle.com/en/java/javase/21/docs/api/index.html
- **ADO.NET Documentation:**
  - Microsoft Docs: https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/
  - Npgsql Documentation: https://www.npgsql.org/doc/
  - System.Data Namespace: https://learn.microsoft.com/en-us/dotnet/api/system.data
- **PostgreSQL:**
  - Official Documentation: https://www.postgresql.org/docs/
  - SQL Tutorial: https://www.postgresql.org/docs/current/tutorial.html